



КУРСОВА РАБОТА

*по дисциплината
„Програмни езици“*

Изготвил: Петър Жан Иванов

Гр. 44^б Фак. №: 121223054

Tech Warehouse

GitHub Repository:

<https://github.com/PetarIvanov33/TechWarehouse>

УВОД

Управлението на складови наличности е основен проблем при разработката на софтуерни системи, свързани с търговия и организация на ресурси. При липса на структурирана система за управление на продукти и количества често възникват грешки, дублиране на данни и затруднения при проследяване на наличностите. Това налага създаването на надеждно решение, което да осигури яснота, контрол и устойчивост на информацията.

Настоящият проект представлява конзолно приложение на езика C++, предназначено за управление на технически склад. Системата позволява добавяне, търсене, редактиране и премахване на продукти, както и управление на количествата чрез операции за увеличаване и намаляване на наличностите. Всеки продукт се идентифицира чрез уникален сериен номер, което гарантира коректна работа и бърз достъп до информацията.

Данните се съхраняват във външен JSON файл, което позволява запазване на състоянието на склада между отделните стартирания на приложението. Реализацията използва обектно-ориентиран подход и включва проверки за валидност на входните данни, с цел предотвратяване на некоректни операции. По този начин проектът решава основните проблеми при управлението на складови наличности и демонстрира практическо приложение на обектно-ориентираното програмиране.

ИЗПОЛЗВАНИ ТЕХНОЛОГИИ И ИНСТРУМЕНТИ

- **C++** – основен език за разработка на приложението
- **Обектно-ориентирано програмиране (ООР)** – класове, наследяване и полиморфизъм
- **STL (Standard Template Library)** – вектори, умни указатели и потоци
- **JSON формат** – съхранение и зареждане на данни
- **nlohmann/json** – външна библиотека за работа с JSON файлове
- **Microsoft Visual Studio** – среда за разработка и дебъгване
- **Git** – система за контрол на версиите

АРХИТЕКТУРА И КЛАСОВА ДИАГРАМА

Приложението е изградено на базата на обектно-ориентирана архитектура, при която всеки основен елемент от системата е представен чрез отделен клас с ясно дефинирана отговорност. Архитектурата следва принципа на разделяне на логиката, като бизнес функционалността, управлението на данните и потребителският интерфейс са логически разграничени.

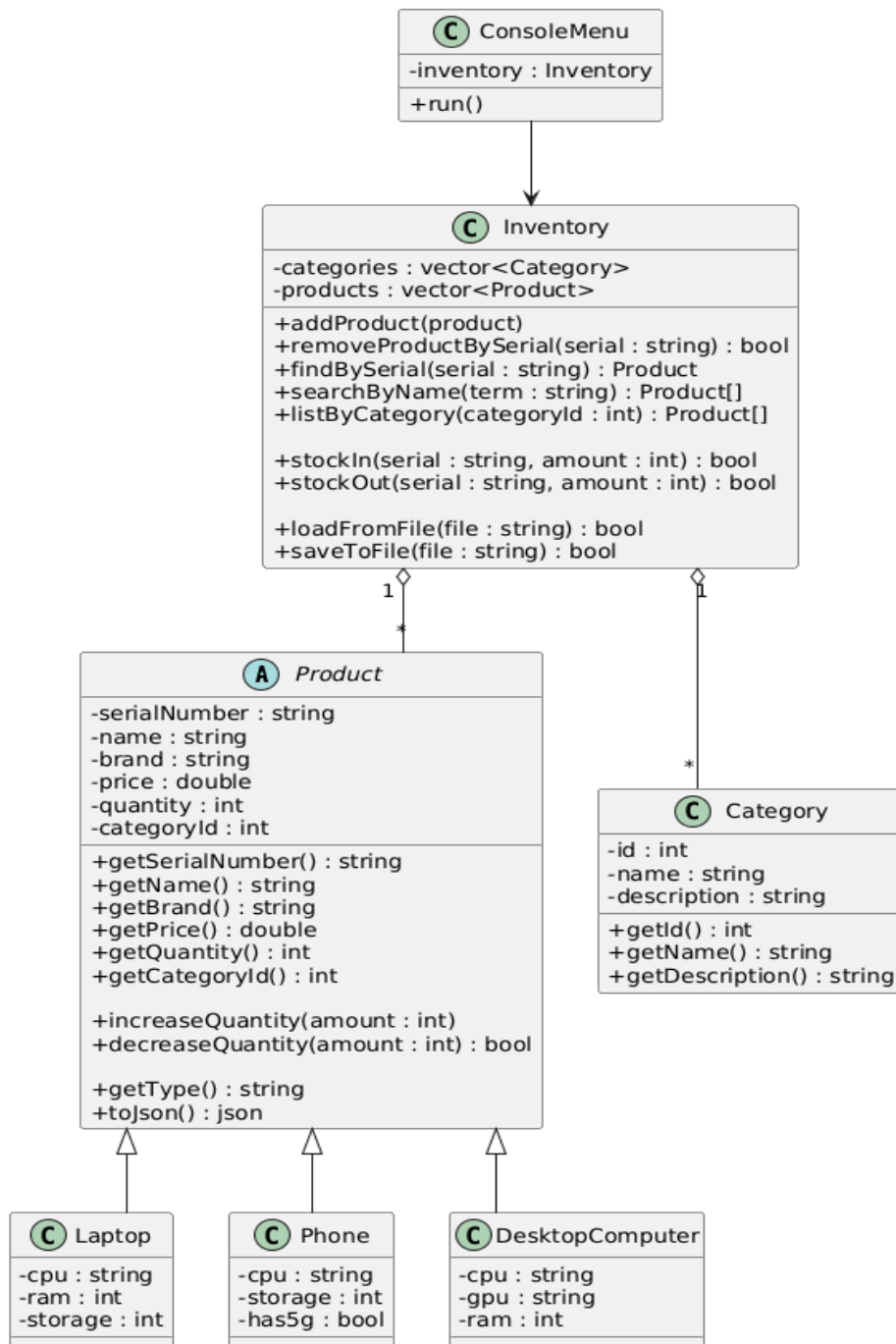
В центъра на системата се намира класът **Inventory**, който отговаря за управлението на всички налични продукти и категории. Този клас съдържа колекции от продукти и категории и предоставя методи за добавяне, търсене, редактиране, изтриване и управление на складовите количества. Освен това, **Inventory** реализира логиката за зареждане и запис на данните във външен JSON файл, което осигурява персистентност на информацията.

Класът **Product** представлява абстрактен базов клас, който дефинира общите характеристики на всички продукти в системата, като сериен номер, име, марка, цена, количество и категория. От него наследяват конкретните класове **Laptop**, **Phone** и **DesktopComputer**, които добавят специфични за съответния тип продукт свойства. Чрез използването на наследяване и полиморфизъм системата позволява работа с различни типове продукти по унифициран начин.

Класът **Category** описва продуктовите категории и съдържа информация за идентификатор, име и описание на категорията. Всеки продукт е свързан с точно една категория чрез идентификатор, което позволява групиране и извеждане на продукти по категории.

Потребителският интерфейс е реализиран чрез класа **ConsoleMenu**, който осигурява взаимодействието между потребителя и системата. Този клас обработва входа от конзолата, визуализира менюто и извиква съответните методи от класа **Inventory**. По този начин се постига ясно разделение между интерфейса и бизнес логиката, като **ConsoleMenu** не съдържа директна логика за обработка на данни.

На фигурата по-долу е представена класова диаграма на системата, която илюстрира връзките между основните класове, наследяването между базовия клас **Product** и неговите производни, както и асоциациите между **Inventory**, **Product**, **Category** и **ConsoleMenu**.



Работа с приложението – потребителско ръководство

1) Стартиране на приложението

1. Отворете проекта във Visual Studio.
2. Стартирайте програмата (**Local Windows Debugger**).
3. При стартиране приложението зарежда данните от файла **warehouse.json** (ако файлът съществува и е валиден).
Ако файлът липсва или е повреден, се започва с празен списък продукти.

След стартиране се визуализира главно меню с наличните операции.

2) Основна навигация

- Всяка операция се избира чрез въвеждане на номер от менюто.
 - При невалиден вход (букви вместо числа, празни стойности, отрицателни количества и т.н.) приложението извежда ясно съобщение за грешка и връща потребителя обратно в менюто.
 - Данните **не се губят**, ако се използва опцията **Save data**, или ако програмата записва автоматично при изход (според настройката в `main`).
-

3) Описание на функциите от менюто

3.1 Add product (Добавяне на продукт)

Служи за добавяне на нов продукт в склада.

Стъпки:

1. Въведете **сериен номер** (уникален).
 - Ако серийният номер вече съществува → операцията се отказва.
2. Въведете **име, марка, цена, начално количество**.
3. Изберете **категория** от списък (Laptop / Phone / Desktop).
4. В зависимост от избраната категория се въвеждат и специфични характеристики:
 - **Laptop**: CPU, RAM(GB), Storage(GB)
 - **Phone**: CPU, Storage(GB), 5G (да/не)

- **DesktopComputer:** CPU, GPU, RAM(GB)

Резултат: продуктът се добавя и вече се вижда при извеждане/търсене.

3.2 List all products (Списък на всички продукти)

Извежда всички продукти със **подробна информация** за всеки:

- Serial, Type, Name, Brand, Price, Quantity, Category
 - специфични полета (CPU/RAM/Storage/GPU/5G) според типа

Полезно за бърза проверка на наличности и данни.

3.3 Search product by NAME (Търсене по име)

Търси продукти по част от името (например "ASUS" или "Galaxy").

Стъпки:

1. Въведете текст за търсене.
2. Програмата извежда всички съвпадения с подробна информация.

Ако няма резултати → извежда се съобщение „Няма намерени продукти“.

3.4 Search product by SERIAL (Търсене по сериен номер)

Търси конкретен продукт по **точен сериен номер**.

Стъпки:

1. Въведете сериен номер.
 2. Ако продуктът съществува → показва се подробен "принт" на продукта.
 3. Ако не съществува → извежда се „Product not found“.
-

3.5 Edit product (Редактиране на продукт)

Позволява редактиране на продукт чрез намиране по сериен номер.

Стъпки:

1. Въведете сериен номер на продукта.
2. Показва се меню за избор на поле за редакция (например име, марка, цена, количество, категория).
3. Въведете новата стойност.

4. При невалидна стойност (примерно отрицателна цена/количество) → редакцията се отказва и се изписва причината.

Важно:

- При промяна на серийния номер се проверява дали новият номер не е зает от друг продукт.
 - Категорията се избира от списък.
-

3.6 Remove product (Изтриване на продукт)

Премахва продукт от склада по сериен номер.

Стъпки:

1. Въведете сериен номер.
 2. Ако продуктът съществува → изтрива се.
 3. Ако продуктът не съществува → извежда се „Not found“.
-

3.7 List products by CATEGORY (Списък по категория)

Извежда продуктите само от избрана категория.

Стъпки:

1. Изберете категория от списък.
 2. Показват се всички продукти от тази категория с подробни данни.
 3. Ако в категорията няма продукти → извежда се съответно съобщение.
-

3.8 Stock IN (Увеличаване на наличност)

Увеличава количеството на съществуващ продукт (например при доставка).

Стъпки:

1. Въведете сериен номер.
 - Ако не съществува → съобщение за грешка.
2. Въведете количество (трябва да е > 0).
 - Ако е невалидно → съобщение за грешка.
3. Количеството се увеличава.

Резултат: показва се новото количество.

3.9 Stock OUT (Намаляване на наличност)

Намалява количеството на продукт (например при продажба/изписване).

Стъпки:

1. Въведете сериен номер.
 - Ако не съществува → съобщение за грешка.
 2. Въведете количество (трябва да е > 0).
 3. Проверка за достатъчна наличност:
 - Ако исканото количество е повече от наличното → съобщение „Not enough quantity“ + наличната стойност.
 4. Количеството се намалява.
-

3.10 List categories (Списък на категории)

Извежда всички налични категории (ID + име).

Категориите са предварително дефинирани (Laptop, Phone, Desktop) и се използват при добавяне/филтриране.

3.11 Save data (Запазване)

Записва текущото състояние на склада във файла **warehouse.json**.

Препоръка: след серия промени (добавяне/редактиране/stock операции) използвайте Save, за да сте сигурни, че данните са записани.

4) Валидации и типични грешки

Приложението валидира всички входни данни, за да не допуска некоректни състояния:

- **Дублиран сериен номер** → добавянето/редакцията се отказва.
- **Празни стойности** (име/серия) → операцията се отказва.
- **Цена ≤ 0** → отказ.
- **Количество < 0** → отказ.
- **Stock OUT с количество $>$ наличното** → отказ + показване на наличното количество.
- **Несъществуващ сериен номер** при търсене/stock/edit/remove → съобщение + връщане към менюто.

Описание на реализацията

1) Общ преглед (как е “сглобено” приложението)

Приложението е конзолна система за управление на склад за техника, разделена логически на 3 части:

- **Модел (данни):** Category, Product + наследници (Laptop, Phone, DesktopComputer)
- **Бизнес логика:** Inventory – пази всички категории и продукти и изпълнява реалните операции (търсене, stock, save/load и т.н.)
- **Потребителски интерфейс (конзола):** ConsoleMenu – показва меню, валидира входа от потребителя и извиква методите на Inventory

TechWarehouse.cpp (main) само стартира процеса: зареждане → меню → запис.

2) Модел на данните (класове)

2.1 Category

Category описва фиксираните категории (Laptop/Phone/Desktop).
Inventory ги създава в конструктора, за да са налични винаги.

2.2 Product и наследяване

Product е абстрактен базов клас за всички продукти, който съдържа общите полета:

- serialNumber (уникален)
- name, brand
- price
- quantity
- categoryId

Наследниците добавят специфични параметри:

- Laptop: CPU, RAM, Storage

- Phone: CPU, Storage, 5G
- DesktopComputer: CPU, GPU, RAM

Ключово: всеки наследник реализира:

- `getType()` – връща типа (например "Laptop")
- `toJson()` – сериализация към JSON
- `fromJson()` – десериализация от JSON

.

3) Inventory.cpp – бизнес логика и работа с данните

3.1 Helper функция: `getStringAny(...)`

```
static std::string getStringAny(const json& j, std::initializer_list<const char*> keys)
{
    for (auto k : keys)
    {
        if (j.contains(k) && j[k].is_string())
            return j[k].get<std::string>();
    }
    return "";
}
```

Какво прави:

Това е помощна функция за “по-толерантно” четене от JSON. Понякога ключовете в JSON може да са изписани по различен начин (`serialNumber`, `serial`, `SerialNumber` и т.н.). Функцията обхожда списък от ключове и връща първия, който съществува и е string.

3.2 Конструктор: `Inventory::Inventory()`

```
Inventory::Inventory() {
    categories.emplace_back(1, "Laptop", "Portable computers");
    categories.emplace_back(2, "Phone", "Mobile phones");
    categories.emplace_back(3, "Desktop", "Desktop computers");
}
```

Какво прави:

Създава фиксираните категории при стартиране. Така приложението винаги има налични категории, без нужда от “AddCategory” функционалност.

3.3 Категории: `getCategories()` и `getCategoryById(...)`

```
const std::vector<Category>& Inventory::getCategories() const {  
    return categories;  
}
```

```
const Category* Inventory::getCategoryById(int id) const {  
    for (const auto& c : categories) {  
        if (c.getId() == id)  
            return &c;  
    }  
    return nullptr;  
}
```

Какво правят:

- `getCategories()` връща целия списък категории (read-only reference).
 - `getCategoryById()` търси категория по ID и връща указател към нея или `nullptr`, ако не съществува.
-

3.4 Добавяне на продукт: `addProduct(...)`

```
void Inventory::addProduct(const std::shared_ptr<Product>& product) {  
    if (!product)  
        return;  
  
    if (serialExists(product->getSerialNumber()))  
        return;  
  
    products.push_back(product);  
}
```

Какво прави:

Добавя продукт в склада, но с две защиты:

- ако е `nullptr` – игнорира

- ако сериалният номер вече съществува – не добавя (уникалност на serial)

3.5 Проверка за уникалност: `serialExists(...)`

```
bool Inventory::serialExists(const std::string& serial) const {
    if (serial.empty())
        return false;

    for (const auto& p : products) {
        if (p->getSerialNumber() == serial)
            return true;
    }
    return false;
}
```

Какво прави:

Връща `true`, ако има продукт със същия сериен номер. Това е основната защита срещу дублиране на продукти.

3.6 Търсене по сериен номер: `findBySerial(...)`

```
std::shared_ptr<Product> Inventory::findBySerial(const std::string& serial)
const {
    if (serial.empty())
        return nullptr;

    for (const auto& p : products) {
        if (p->getSerialNumber() == serial)
            return p;
    }
    return nullptr;
}
```

Какво прави:

Връща продукта (като `shared_ptr`) или `nullptr`, ако няма такъв сериен номер. Използва се масово от менюто за валидации и операции.

3.7 Всички продукти: `getAllProducts()`

```
std::vector<std::shared_ptr<Product>> Inventory::getAllProducts() const {
    return products;
}
```

```
}
```

Какво прави:

Връща копие на вектора с продукти. Това е удобен “read” метод за ConsoleMenu.

3.8 Търсене по име: `searchByName( . . . )`

```
std::vector<std::shared_ptr<Product>> Inventory::searchByName(const std::string&
term) const {
    std::vector<std::shared_ptr<Product>> result;

    if (term.empty())
        return result;

    for (const auto& p : products) {
        if (p->getName().find(term) != std::string::npos) {
            result.push_back(p);
        }
    }
    return result;
}
```

Какво прави:

Търси продукти, чийто `name` съдържа подниз `term`. Връща списък от съвпадения (може да са много).

3.9 Филтър по категория: `listByCategory( . . . )`

```
std::vector<std::shared_ptr<Product>> Inventory::listByCategory(int categoryId)
const {
    std::vector<std::shared_ptr<Product>> result;

    if (!getCategoryById(categoryId))
        return result;

    for (const auto& p : products) {
        if (p->getCategoryId() == categoryId) {
            result.push_back(p);
        }
    }
}
```

```
    return result;
}
```

Какво прави:

Връща всички продукти от избрана категория. Първо проверява дали категорията съществува, за да не филтрира по невалидно ID.

3.10 Премахване по сериен номер: `removeProductBySerial(...)`

```
bool Inventory::removeProductBySerial(const std::string& serial) {
    if (serial.empty())
        return false;

    auto it = std::remove_if(
        products.begin(),
        products.end(),
        [&](const std::shared_ptr<Product>& p) {
            return p->getSerialNumber() == serial;
        }
    );

    if (it == products.end())
        return false;

    products.erase(it, products.end());
    return true;
}
```

Какво прави:

Изтрива продукт по сериен номер. Използва `std::remove_if` + `erase`, което е стандартния C++ шаблон за изтриване от `vector`.

Връща `true` ако е изтрит, `false` ако не е намерен.

3.11 Редакция “чрез JSON”: `updateProductFromJson(...)`

```
bool Inventory::updateProductFromJson(const std::string& currentSerial, const
json& updatedJson)
{
```

```

    auto existing = findBySerial(currentSerial);
    if (!existing)
        return false;

    std::string newSerial = getStringAny(updatedJson, { "serialNumber",
"serial", "SerialNumber", "Serial" });
    if (newSerial.empty())
        newSerial = existing-&gtgetSerialNumber();

    if (newSerial != currentSerial && serialExists(newSerial))
        return false;

    std::string type = getStringAny(updatedJson, { "type", "Type" });
    if (type.empty())
        type = existing-&gtgetType();

    std::shared_ptr<Product> newP;

    try
    {
        if (type == "Laptop")
            newP = std::make_shared<Laptop>(Laptop::fromJson(updatedJson));
        else if (type == "Phone")
            newP = std::make_shared<Phone>(Phone::fromJson(updatedJson));
        else if (type == "DesktopComputer")
            newP =
std::make_shared<DesktopComputer>(DesktopComputer::fromJson(updatedJson));
        else
            return false;
    }
    catch (...)
    {
        return false;
    }

    for (auto& p : products)
    {

```

```

        if (p && p->getSerialNumber() == currentSerial)
        {
            p = newP;
            return true;
        }
    }

    return false;
}

```

Какво прави (важното):

1. Намира продукта по текущ serial. Ако няма – край.
2. Чете нов serial от JSON (с толеранс към имена на ключове).
3. Проверява за дублиране на serial.
4. Чете типа (Laptop/Phone/DesktopComputer) и създава **нов обект** чрез съответния fromJson(. . .).
5. Заменя стария shared_ptr във вектора с новия.

Защо е направено така:

Понеже в моделите няма публични setter-и за всички полета, този подход позволява **пълна редакция** без промяна по класовете Product/Laptop/Phone/DesktopComputer.

3.12 Валидации за stock: canStockIn(. . .) и canStockOut(. . .)

```

bool Inventory::canStockIn(const std::string& serial, int amount) const {
    if (serial.empty())
        return false;

    if (amount <= 0)
        return false;

    auto product = findBySerial(serial);
    if (!product)
        return false;

    return true;
}

```



```

bool Inventory::canStockOut(const std::string& serial, int amount) const {
    if (serial.empty())
        return false;

    if (amount <= 0)
        return false;

    auto product = findBySerial(serial);
    if (!product)
        return false;

    if (amount > product->getQuantity())
        return false;

    return true;
}

```

Какво правят:

Това са проверки преди реална промяна на quantity.

canStockOut включва и проверка да не се извади повече от наличното.

3.13 Реални stock операции: stockIn(. . .) и stockOut(. . .)

```

bool Inventory::stockIn(const std::string& serial, int amount) {
    if (!canStockIn(serial, amount))
        return false;

    auto product = findBySerial(serial);
    product->increaseQuantity(amount);
    return true;
}

bool Inventory::stockOut(const std::string& serial, int amount) {
    if (!canStockOut(serial, amount))
        return false;
}

```

```
    auto product = findBySerial(serial);  
    return product->decreaseQuantity(amount);  
}
```

Какво правят:

Извършват промяната “дефанзивно” – първо викат `canStock...` и чак тогава действат. Това гарантира, че дори ако UI се опита да подаде грешни данни, бизнес логиката няма да позволи.

3.14 Зареждане от JSON: `loadFromFile(...)`

```
bool Inventory::loadFromFile(const std::string& file) {  
    std::ifstream in(file);  
    if (!in.is_open())  
        return false;  
  
    json j;  
    try {  
        in >> j;  
    }  
    catch (...) {  
        return false;  
    }  
  
    products.clear();  
  
    if (!j.contains("products") || !j["products"].is_array())  
        return true;  
  
    for (const auto& item : j["products"]) {  
        if (!item.contains("type"))  
            continue;  
  
        try {  
            std::string type = item["type"].get<std::string>();  
  
            if (type == "Laptop") {  
  
products.push_back(std::make_shared<Laptop>(Laptop::fromJson(item)));
```

```

        }
        else if (type == "Phone") {

products.push_back(std::make_shared<Phone>(Phone::fromJson(item)));

        }
        else if (type == "DesktopComputer") {

products.push_back(std::make_shared<DesktopComputer>(DesktopComputer::fromJson(item)));

        }
    }
    catch (...) {
        continue;
    }
}

return true;
}

```

Какво прави:

- Отваря файла и парсва JSON
- Чисти текущите продукти
- Обхожда масива `products`
- По полето `"type"` определя конкретния клас и създава обект чрез `fromJson(item)`
- При повреден елемент – пропуска го, без да спира програмата

3.15 Запис в JSON: `saveToFile(...)`

```

bool Inventory::saveToFile(const std::string& file) {
    json j;
    j["products"] = json::array();

    for (const auto& p : products) {
        j["products"].push_back(p->toJson());
    }

    std::ofstream out(file);
    if (!out.is_open())

```

```
        return false;

    out << j.dump(4);
    return true;
}
```

Какво прави:

- Създава JSON с ключ "products"
- Всеки продукт се сериализира с `toJson()` (полиморфно)
- Записва файла в pretty формат (`dump(4)`)

4) ConsoleMenu.cpp – интерфейс, UX, валидации и управление на операциите

4.1 Входна помощна функция: `clearInput()`

```
static void clearInput()
{
    std::cin.clear();
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}
```

Какво прави:

Чисти `std::cin` при грешен вход (например букви вместо число) и изпразва остатъка от реда. Това е задължително за стабилно меню.

4.2 Цветове в конзолата (Windows): `initConsoleColors`, `setColor`, `resetColor`

```
#ifdef _WIN32
static WORD g_defaultAttr = 0;
static bool g_inited = false;

static void initConsoleColors()
{
    if (g_inited) return;
    g_inited = true;
}
```

```

HANDLE h = GetStdHandle(STD_OUTPUT_HANDLE);
CONSOLE_SCREEN_BUFFER_INFO info;
if (GetConsoleScreenBufferInfo(h, &info))
    g_defaultAttr = info.wAttributes;
else
    g_defaultAttr = FOREGROUND_RED | FOREGROUND_GREEN | FOREGROUND_BLUE;
}

static void setColor(WORD attr)
{
    initConsoleColors();
    SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), attr);
}

static void resetColor()
{
    initConsoleColors();
    SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), g_defaultAttr);
}

#else
static void setColor(int) {}
static void resetColor() {}
#endif

```

Какво прави:

На Windows взима “дефолтния” цвят на конзолата и позволява временно оцветяване на текст. На не-Windows платформи функциите са празни, за да компилира навсякъде.

4.3 UX принтове: `printTitle`, `printOk`, `printError`, `printInfo`

Пример:

```

static void printOk(const std::string& text)
{
#ifdef _WIN32
    setColor(FOREGROUND_GREEN | FOREGROUND_INTENSITY);
#endif

```

```

std::cout << "[OK] " << text << "\n";
resetColor();
}

```

Какво правят:

Стандартизирант съобщенията, за да изглежда приложението по-подредено:

- printTitle – секция/екран
- printOk – успех
- printError – грешка
- printInfo – неутрална информация

4.4 Подробен принт на продукт: printProductDetailed(. . .)

```

static void printProductDetailed(const Inventory& inv, const
std::shared_ptr<Product>& p)
{
    if (!p) return;

    const Category* cat = inv.getCategoryById(p->getCategoryId());
    std::string catName = cat ? cat->getName() : "Unknown";

    std::cout << " Serial    : " << p->getSerialNumber() << "\n";
    std::cout << " Type      : " << p->getType() << "\n";
    std::cout << " Name      : " << p->getName() << "\n";
    std::cout << " Brand     : " << p->getBrand() << "\n";
    std::cout << " Price     : " << p->getPrice() << "\n";
    std::cout << " Quantity  : " << p->getQuantity() << "\n";
    std::cout << " Category : " << catName << " (ID=" << p->getCategoryId() <<
    ") \n";
}

```

Какво прави:

Единен “шаблон” за показване на продукт. Използва се в:

- list all
- search by name/serial
- list by category
- edit preview

Така навсякъде изглежда еднакво и ясно.

4.5 Избор на категория: `chooseCategoryId( . . . )`

```
static int chooseCategoryId(const Inventory& inv)
{
    const auto& cats = inv.getCategories();
    if (cats.empty())
    {
        printError("No categories available.");
        return -1;
    }

    printTitle("CHOOSE CATEGORY");
    for (size_t i = 0; i < cats.size(); i++)
        std::cout << (i + 1) << ". " << cats[i].getName() << "\n";

    std::cout << "Choice (1-" << cats.size() << "): ";
    int choice;
    std::cin >> choice;

    if (std::cin.fail())
    {
        clearInput();
        printError("Invalid input.");
        return -1;
    }

    clearInput();

    if (choice < 1 || choice > (int)cats.size())
    {
        printError("Invalid category choice.");
        return -1;
    }

    return cats[choice - 1].getId();
}
```

```
}
```

Какво прави:

Потребителят не пише ID на категорията ръчно. Избира от меню 1..N и се връща реалния categoryId.

4.6 Конструктор на ConsoleMenu

```
ConsoleMenu::ConsoleMenu(Inventory& inventory)
    : inventory(inventory)
{
#ifdef _WIN32
    initConsoleColors();
#endif
}
```

Какво прави:

Запазва референция към Inventory и подготвя конзолни цветове (Windows).

4.7 Показване на меню: showMenu()

```
void ConsoleMenu::showMenu() const
{
    printTitle("TECH WAREHOUSE");
    // ... опции ...
    std::cout << "Choice: ";
}
```

Какво прави:

Отпечатва главното меню със всички операции.

4.8 Главен цикъл: run()

```
void ConsoleMenu::run()
{
    int choice = -1;

    while (choice != 0)
    {
        showMenu();
        std::cin >> choice;
    }
}
```



```

        if (std::cin.fail())
        {
            clearInput();
            printError("Invalid input.");
            continue;
        }

        clearInput();

        switch (choice)
        {
        case 1: addProduct(); break;
        // ...
        case 0: printOk("Exiting..."); break;
        default: printError("Unknown option."); break;
        }
    }
}

```

Какво прави:

Това е основният “engine” на UI:

- показва менюто
- чете избор
- валидира входа
- извиква конкретния метод

4.9 Списък категории: `listCategories()`

```

void ConsoleMenu::listCategories()
{
    const auto& cats = inventory.getCategories();
    if (cats.empty())
    {
        printError("No categories.");
        return;
    }
}

```

```

    printTitle("CATEGORIES");
    for (const auto& c : cats)
        std::cout << c.getId() << " | " << c.getName() << "\n";
}

```

Какво прави:

Показва всички налични категории (ID + име).

4.10 Списък продукти: `listProducts()`

```

void ConsoleMenu::listProducts()
{
    const auto& products = inventory.getAllProducts();
    if (products.empty())
    {
        printInfo("No products.");
        return;
    }

    printTitle("ALL PRODUCTS");
    for (const auto& p : products)
        printProductDetailed(inventory, p);
}

```

Какво прави:

Извежда всички продукти с подробен принт.

4.11 Добавяне на продукт: `addProduct()`

(важни части – примерни блокове)

Въвеждане и валидация на основни данни:

```

std::cout << "Serial number: ";
std::getline(std::cin, serial);

if (serial.empty()) { printError("Serial number cannot be empty."); return; }
if (inventory.findBySerial(serial)) { printError("Product with this serial
already exists."); return; }

```

Какво прави:

Гарантира, че:

- serial не е празен
- serial е уникален

Избор на категория (само веднъж):

```
int categoryId = chooseCategoryId(inventory);  
if (categoryId == -1) return;
```

Какво прави:

Потребителят избира категория, а оттам автоматично се определя типът продукт.

Автоматично определяне на тип според категорията:

```
if (categoryId == 1 || catName == "Laptop") { ... Laptop ... }  
else if (categoryId == 2 || catName == "Phone") { ... Phone ... }  
else if (categoryId == 3 || catName == "Desktop") { ... DesktopComputer ... }
```

Какво прави:

Премахва нуждата да се избира “тип” втори път, защото категорията и типът са едно и също в проекта.

4.12 Търсене по име: `searchByName()`

```
void ConsoleMenu::searchByName()  
{  
    std::string term;  
    printTitle("SEARCH BY NAME");  
  
    std::cout << "Name: ";  
    std::getline(std::cin, term);  
  
    auto results = inventory.searchByName(term);  
    if (results.empty())  
    {  
        printInfo("No matches.");  
        return;  
    }  
  
    for (const auto& p : results)  
        printProductDetailed(inventory, p);  
}
```

Какво прави:

Връща всички съвпадения и ги печата подробно.

4.13 Търсене по сериен номер: `searchBySerial()`

```
void ConsoleMenu::searchBySerial()
{
    std::string serial;
    printTitle("SEARCH BY SERIAL");

    std::cout << "Serial: ";
    std::getline(std::cin, serial);

    auto p = inventory.findBySerial(serial);
    if (!p)
    {
        printError("Product not found.");
        return;
    }

    printProductDetailed(inventory, p);
}
```

Какво прави:

Намира 1 продукт и го печата подробно.

4.14 Редакция на продукт: `editProduct()`

Идеята: взима се `toJson()`, променят се полета, после се вика `updateProductFromJson()`.

```
auto j = p->toJson();
// ... редакции по j["name"], j["price"], j["cpu"] ...
if (!inventory.updateProductFromJson(serial, j))
{
    printError("Edit failed (duplicate serial or invalid data).");
    return;
}
```

Какво прави:

Позволява редакция на:

- общи полета (serial, name, brand, price, quantity, category)

- специфични според типа (cpu/ram/storage/gpu/5g)

Важно: приложението пази ограничения:

- serial да не се дублира
- цена > 0, quantity >= 0, ram/storage > 0 и т.н.

4.15 Изтриване: `removeProduct()`

```
void ConsoleMenu::removeProduct()
{
    printTitle("REMOVE PRODUCT");

    std::string serial;
    std::cout << "Serial: ";
    std::getline(std::cin, serial);

    if (!inventory.removeProductBySerial(serial))
        printError("Not found.");
    else
        printOk("Removed.");
}
```

Какво прави:

Премахва продукт по serial и показва ясно дали е намерен.

4.16 Списък по категория: `listByCategory()`

```
void ConsoleMenu::listByCategory()
{
    int categoryId = chooseCategoryId(inventory);
    if (categoryId == -1) return;

    auto products = inventory.listByCategory(categoryId);
    if (products.empty())
    {
        printInfo("No products in this category.");
        return;
    }
}
```

```

        printTitle("PRODUCTS BY CATEGORY");
        for (const auto& p : products)
            printProductDetailed(inventory, p);
    }

```

Какво прави:

Избор на категория от меню → показва продуктите вътре (подробно).

4.17 Stock IN: stockIn()

```

auto p = inventory.findBySerial(serial);
if (!p) { printError("Invalid serial number (product not found)."); return; }

std::cin >> amount;
if (std::cin.fail()) { clearInput(); printError("Invalid amount."); return; }
if (amount <= 0) { printError("Amount must be > 0."); return; }

inventory.stockIn(serial, amount);
printOk("Stock updated.");
printInfo("New quantity: " + std::to_string(p->getQuantity()));

```

Какво прави:

- проверява дали продуктът съществува
 - валидира amount (число, > 0)
 - увеличава quantity и показва новата стойност
-

4.18 Stock OUT: stockOut()

```

auto p = inventory.findBySerial(serial);
if (!p) { printError("Invalid serial number (product not found)."); return; }

std::cin >> amount;
if (std::cin.fail()) { clearInput(); printError("Invalid amount."); return; }
if (amount <= 0) { printError("Amount must be > 0."); return; }

if (amount > p->getQuantity())
{
    printError("Not enough quantity.");
    printInfo("Available: " + std::to_string(p->getQuantity()));
}

```

```

        return;
    }

    inventory.stockOut(serial, amount);
    printOk("Stock updated.");
    printInfo("New quantity: " + std::to_string(p->getQuantity()));

```

Какво прави:

Същото като Stock IN, но има допълнителна проверка да не се извади повече от наличното.

4.19 Запис: `saveData()`

```

void ConsoleMenu::saveData()
{
    if (inventory.saveToFile("warehouse.json"))
        printOk("Data saved.");
    else
        printError("Save failed.");
}

```

Какво прави:

Запазва текущите продукти във `warehouse.json` и показва резултат.

Валидации и защита

Проектът е направен така, че да е **устойчив на грешен вход** и да **не допуска нелогични операции**, които биха развалили данните в склада.

Основни защиты и проверки:

- **Сериен номер**
 - не може да е празен;
 - не се допуска дублиране (проверка чрез `Inventory::serialExists()` / `findBySerial()`).
- **Числови стойности**
 - `price` трябва да е > 0 ;
 - `quantity` трябва да е ≥ 0 ;
 - `stock in/out amount` трябва да е > 0 .
- **Stock OUT ограничения**
 - не се допуска изписване на повече бройки от наличните (проверка чрез `Inventory::canStockOut()` и допълнително UI-съобщение в менюто).

- **Съществуване на продукт**
 - при операции като Search / Edit / Remove / Stock се проверява дали серийният номер съществува, иначе се връща контролът към менюто.
- **Категории**
 - изборът е ограничен до реално наличните категории (чрез списък/избор, вместо свободно въвеждане на ID).
- **„Defensive programming“ в Inventory**
 - дори менюто да е проверило входа, Inventory пак валидира критичните операции (пример: `stockIn()` и `stockOut()` използват `canStockIn()` / `canStockOut()`).
- **Edit на продукт**
 - редакцията минава през `updateProductFromJson()`, което:
 - проверява дали продуктът съществува;
 - проверява дали новият serial (ако се променя) не дублира друг продукт;
 - проверява типа и дали JSON е валиден (с `try/catch`).

JSON / персистентност

За да се запазват данните между стартиранията, приложението използва **файл** `warehouse.json` и библиотеката `nlohmann/json`.

Как работи запазването:

- `Inventory::saveToFile(file)`
 - обхожда всички продукти;
 - за всеки продукт извиква `toJson()` (полиморфно – според реалния тип `Laptop/Phone/DesktopComputer`);
 - записва масив `products` във файла във четим формат (`dump(4)`).

Как работи зареждането:

- `Inventory::loadFromFile(file)`
 - отваря файла и чете JSON;
 - проверява дали има масив `products`;
 - за всеки елемент гледа полето `type`;
 - според `type` създава правилния обект чрез `Laptop::fromJson()`, `Phone::fromJson()`, `DesktopComputer::fromJson()`;

- при проблемен/повреден елемент – пропуска го безопасно (try/catch), без да “чупи” приложението.

Така приложението има:

- **устойчивост** при повреден JSON;
- **реална персистентност** (данните остават след затваряне);
- **разширяемост** (лесно се добавя нов тип продукт с нов toJson/fromJson).

Заклучение

TechWarehouse реализира **конзолна система за управление на технически склад**, която покрива основните нужди: добавяне, търсене, редактиране, изтриване, филтриране по категория и управление на наличности (Stock IN/OUT), както и запазване/зареждане на данни чрез JSON.